Predicting Customer Purchase Behaviour Using Apache Spark and Machine Learning

INT315 - Cluster Computing (CA2)

Academic project report

Prepared with measured outputs from the local Spark execution. No unrun results or screenshots are represented as evidence.

Table of Contents

1. Project Title 3

2. Problem Statement 3

3. Objective of the Project 3

4. Dataset Description 3

5. Dataset Source 3

6. Tools and Technologies Used 3

7. Implementation and Methodology 4

8. Complete Source Code 4

9. Execution and Output Screenshots 4

10. Results and Analysis 5

11. Conclusion 7

12. Plagiarism Report 7

13. References 8

Appendix A - Complete Python Source Code 9

1. Project Title

Predicting Customer Purchase Behaviour Using Apache Spark and Machine Learning.

2. Problem Statement

E-commerce teams need to understand which browsing sessions are associated with a purchase. This project uses session-level website interaction data to classify whether a session ends with a transaction. The target is binary: 1 indicates that a purchase was recorded and 0 indicates no purchase.

3. Objective of the Project

The objective is to build a reproducible Spark workflow that loads and validates the public dataset, performs leakage-aware preprocessing, trains Logistic Regression and Gradient-Boosted Trees, evaluates the models on a held-out test set, exports session predictions, and presents the measured outputs in a dashboard.

4. Dataset Description

The dataset contains online-shopping sessions rather than demographic customer profiles. The public UCI record describes 12,330 sessions, with one session per user in the sampled period. The project keeps the source features and uses a generated session identifier only for output traceability; that identifier is not a source customer ID and is excluded from training. The local project copy used for this run is customer_behaviour.csv.

Field(s)	Meaning
Administrative, Informational, ProductRelated	Counts of visited pages in each page category.
*_Duration	Time spent in each corresponding page category during the session.
BounceRates, ExitRates, PageValues	Web analytics measures associated with the session.
SpecialDay	Proximity measure for a special shopping day.
Month, OperatingSystems, Browser, Region, TrafficType	Session time and technical/traffic categories.
VisitorType, Weekend	New/returning visitor category and weekend indicator.
Revenue -> purchase	Binary outcome; True maps to 1 (purchase), False to 0.

Limitations: the source has no age/age-group, explicit session-duration field, or previous purchase-count field. Session duration in this project is derived from the sum of the three page-category duration features. VisitorType distinguishes new from returning visitors but is not a count of past purchases. The project does not fabricate the missing attributes.

5. Dataset Source

Source: Sakar, C. and Kastro, Y. (2018), Online Shoppers Purchasing Intention Dataset, UCI Machine Learning Repository. DOI: 10.24432/C5F88Q. Dataset page: https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset. Direct archive: https://archive.ics.uci.edu/static/public/468/online+shoppers+purchasing+intention+dataset.zip. The dataset is distributed under CC BY 4.0; attribution is required.

6. Tools and Technologies Used

Python 3, Apache Spark in local[*] mode, PySpark SQL and MLlib, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn for ROC curve coordinates, Streamlit, Plotly, and ReportLab. Local mode uses one machine and is not represented as a multi-node cluster run.

7. Implementation and Methodology

On this execution, 12,330 source rows were loaded and 12,205 remained after validation and duplicate removal. The split used 80% training data and a fixed seed of 42. Numerical medians and categorical modes are estimated from training rows only and applied to both partitions. StringIndexers, OneHotEncoder, VectorAssembler, StandardScaler, and the classifier are fitted together in each Spark ML Pipeline.

Logistic Regression estimates a linear decision boundary for the binary target. Gradient-Boosted Trees combine shallow decision trees sequentially to model nonlinear feature interactions. Both pipelines are fit only on training data and produce predictions for the same held-out test partition.

Observed source target distribution: {'1.0': 1908, '0.0': 10297}. The identifiers used in exported predictions are generated session row IDs, not customer identities from UCI.

8. Complete Source Code

The executable Python sources used for this project are included in the appendices of this PDF. The workspace files remain the authoritative editable source.

9. Execution and Output Screenshots

Authentic screenshots are not generated by the report process. Insert screenshots captured from your own execution in the marked locations below and save the originals under screenshots/.

Screenshot placeholder: Project folder structure
Insert an authentic project screenshot here.

Screenshot placeholder: Dataset loaded into Spark and schema
Insert an authentic project screenshot here.

Screenshot placeholder: Missing-value analysis and preprocessing output
Insert an authentic project screenshot here.

Screenshot placeholder: Feature encoding and VectorAssembler
Insert an authentic project screenshot here.

Screenshot placeholder: Logistic Regression training
Insert an authentic project screenshot here.

Screenshot placeholder: Gradient-Boosted Trees training
Insert an authentic project screenshot here.

Screenshot placeholder: Model evaluation metrics
Insert an authentic project screenshot here.

Screenshot placeholder: Confusion matrix and ROC curve
Insert an authentic project screenshot here.

Screenshot placeholder: Actual-versus-predicted purchases
Insert an authentic project screenshot here.

Screenshot placeholder: Dashboard overview and customer segmentation
Insert an authentic project screenshot here.

Screenshot placeholder: Interactive session prediction
Insert an authentic project screenshot here.

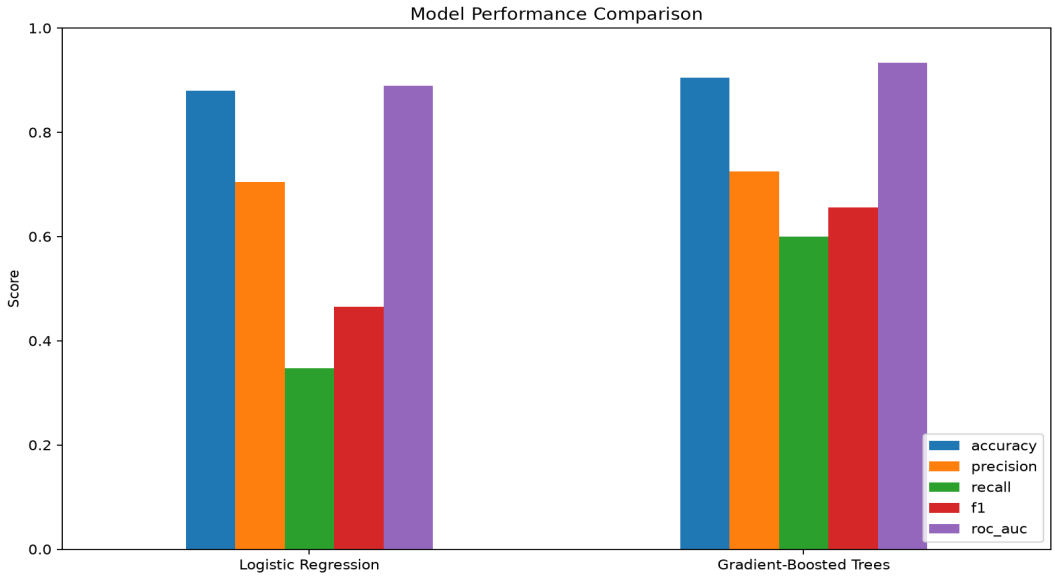
Screenshot placeholder: Successful end-to-end execution
Insert an authentic project screenshot here.

10. Results and Analysis

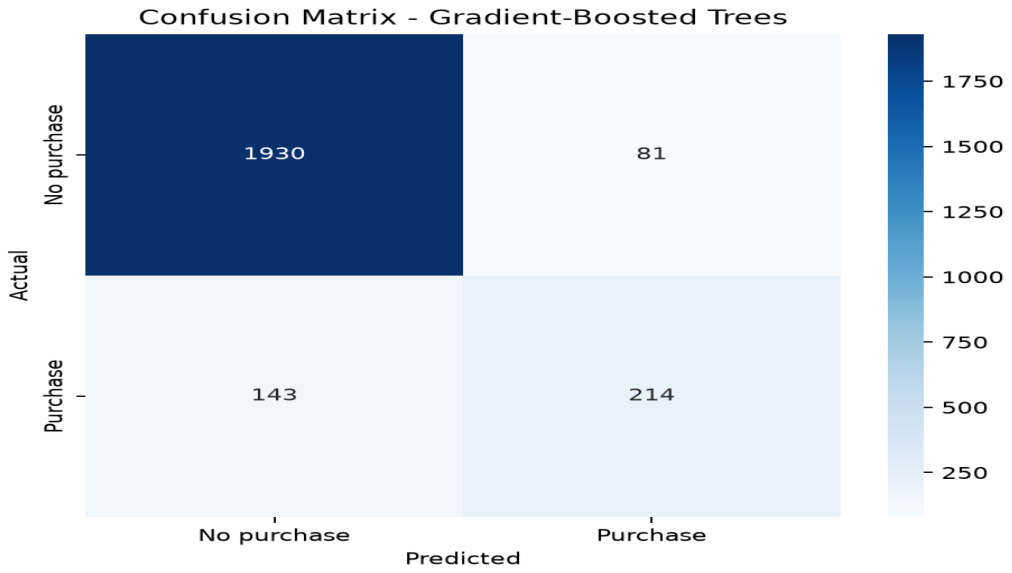
Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.8796	0.7045	0.3473	0.4653	0.8896
Gradient-Boosted Trees	0.9054	0.7254	0.5994	0.6564	0.9338

Selected model: Gradient-Boosted Trees. Selection prioritizes purchase-class F1-score, with ROC-AUC and accuracy as tie-breakers. Accuracy measures the fraction of all correct classifications; precision measures how many predicted purchases were actual purchases; recall measures how many actual purchases were detected; F1 balances precision and recall; ROC-AUC measures ranking quality over decision thresholds. The following values are read from Spark evaluation on this run's holdout predictions and must not be interpreted as guarantees of future performance.

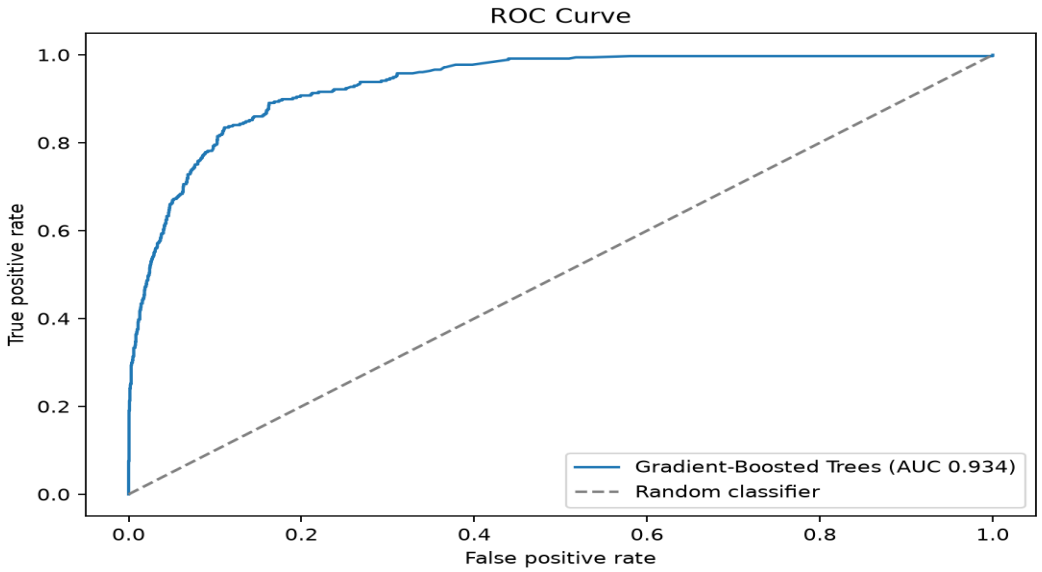
Performance comparison of the two Spark models.



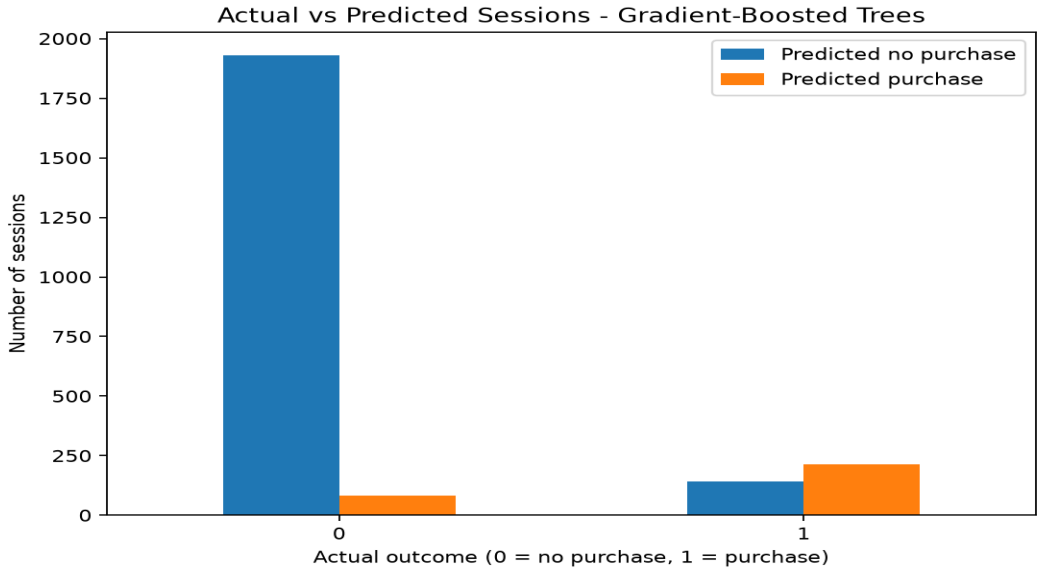
Best-model confusion matrix on the held-out test sessions.



Best-model ROC curve on the held-out test sessions.



Actual and predicted outcome counts for the holdout set.



11. Conclusion

This project demonstrates a complete local Spark ML workflow using a public, documented e-commerce session dataset. The trained models and test metrics are generated by execution, and the Streamlit dashboard reads persisted artifacts rather than retraining on page refresh. Future work may validate generalization on newer sessions, tune thresholds for business costs, and use a consented dataset that actually includes demographic and prior-purchase attributes.

12. Plagiarism Report

A plagiarism/similarity report must be generated separately using a university-approved plagiarism-checking tool. No similarity percentage is asserted in this document. Attach the institutionally generated report here if required.

Screenshot placeholder: University plagiarism-checking report (attach separately)
Insert an authentic project screenshot here.

13. References

Sakar, C. and Kastro, Y. (2018). Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. https://doi.org/10.24432/C5F88Q.

Apache Spark documentation: https://spark.apache.org/docs/latest/

UCI dataset page: https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset

Appendix A - Complete Python Source Code

main.py

```
"""Run the complete local Spark training, evaluation, and reporting workflow."""

from __future__ import annotations

import argparse
import json
import logging
from pathlib import Path

from src.data_loader import DATASET_PATH, DATASET_URL, UCI_DATASET_PAGE, ensure_dataset
from src.data_preprocessing import load_dataset, preprocess_data
from src.model_evaluation import evaluate_models
from src.model_training import train_models
from src.prediction import export_predictions
from src.spark_session import create_spark_session

PROJECT_ROOT = Path(__file__).resolve().parent
OUTPUT_DIRECTORY = PROJECT_ROOT / "outputs"
MODEL_DIRECTORY = PROJECT_ROOT / "models"
REPORT_PATH = PROJECT_ROOT / "report" / "project_report.pdf"


def run_pipeline(dataset_path: Path = DATASET_PATH, refresh_data: bool = False) -> dict:
    """Execute ingestion through PDF report generation and return run artifacts."""
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )
    OUTPUT_DIRECTORY.mkdir(parents=True, exist_ok=True)
    MODEL_DIRECTORY.mkdir(parents=True, exist_ok=True)

    local_dataset = ensure_dataset(dataset_path, overwrite=refresh_data)
    spark = create_spark_session()
    try:
        raw_data = load_dataset(spark, str(local_dataset))
        print("\nLoaded Spark DataFrame schema:")
        raw_data.printSchema()
        print("\nFirst 10 sessions:")
        raw_data.show(10, truncate=False)
        print(f"Rows: {raw_data.count():,} | Columns: {len(raw_data.columns)}")

        training_data, test_data, preprocessing_summary = preprocess_data(raw_data)
        summary_path = OUTPUT_DIRECTORY / "preprocessing_summary.json"
        summary_path.write_text(json.dumps(preprocessing_summary, indent=2), encoding="utf-8")
        print("\nNull values before cleaning:")
        print(json.dumps(preprocessing_summary["null_counts"], indent=2))
        print("\nPurchase class distribution:")
        print(json.dumps(preprocessing_summary["class_distribution"], indent=2))
        print("\nPreprocessed training sample:")
        training_data.show(5, truncate=False)

        trained_models = train_models(
            training_data,
            test_data,
            models_directory=MODEL_DIRECTORY,
            random_seed=42,
        )
        evaluation = evaluate_models(trained_models, OUTPUT_DIRECTORY)
        predictions_path = export_predictions(
            trained_models[evaluation["best_model"]]["model_path"],
            test_data,
        )

        from report.generate_report import generate_report

        report_path = generate_report(
            preprocessing_summary=preprocessing_summary,
            evaluation=evaluation,
            dataset_path=local_dataset,
            output_path=REPORT_PATH,
        )
        return {
            "dataset": str(local_dataset),
            "dataset_source": DATASET_URL,
            "dataset_page": UCI_DATASET_PAGE,
            "metrics": str(OUTPUT_DIRECTORY / "model_metrics.json"),
            "predictions": str(predictions_path),
            "report": str(report_path),
            "best_model": evaluation["best_model"],
        }
    finally:
        spark.stop()


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--dataset-path",
```

```
        type=Path,
        default=DATASET_PATH,
        help="Path to the UCI CSV (downloaded there if it does not exist).",
    )
    parser.add_argument(
        "--refresh-data",
        action="store_true",
        help="Download and validate the official UCI archive again.",
    )
    return parser.parse_args()


if __name__ == "__main__":
    artifacts = run_pipeline(**vars(parse_args()))
    print("\nPipeline completed successfully:")
    print(json.dumps(artifacts, indent=2))
```

src__init__.py

src\data_loader.py

```
"""Download the public UCI Online Shoppers Purchasing Intention dataset."""

from __future__ import annotations

import logging
import re
import zipfile
from io import BytesIO
from pathlib import Path
from urllib.request import Request, urlopen

import pandas as pd

LOGGER = logging.getLogger(__name__)

DATASET_PATH = Path(__file__).resolve().parent.parent / "data" / "customer_behaviour.csv"
DATASET_URL = (
    "https://archive.ics.uci.edu/static/public/468/"
    "online+shoppers+purchasing+intention+dataset.zip"
)
UCI_DATASET_PAGE = (
    "https://archive.ics.uci.edu/dataset/468/"
    "online+shoppers+purchasing+intention+dataset"
)

COLUMN_RENAMES = {
    "administrative": "administrative",
    "administrativeduration": "administrative_duration",
    "informational": "informational",
    "informationalduration": "informational_duration",
    "productrelated": "product_related",
    "productrelatedduration": "product_related_duration",
    "bouncerates": "bounce_rates",
    "exitrates": "exit_rates",
    "pagevalues": "page_values",
    "specialday": "special_day",
    "month": "month",
    "operatingsystems": "operating_systems",
    "browser": "browser",
    "region": "region",
    "traffictype": "traffic_type",
    "visitortype": "visitor_type",
    "weekend": "weekend",
    "revenue": "purchase",
}

NUMERIC_COLUMNS = (
    "administrative",
    "administrative_duration",
    "informational",
    "informational_duration",
    "product_related",
    "product_related_duration",
    "bounce_rates",
    "exit_rates",
    "page_values",
    "special_day",
)
CATEGORICAL_COLUMNS = (
    "month",
    "operating_systems",
    "browser",
    "region",
    "traffic_type",
    "visitor_type",
    "weekend",
)
TARGET_COLUMN = "purchase"
```

```
DATASET_COLUMNS = (*NUMERIC_COLUMNS, *CATEGORICAL_COLUMNS, TARGET_COLUMN)


def _normalize_column_name(column: str) -> str:
    """Convert the dataset's original names into the documented project schema."""
    normalized = re.sub(r"[^a-z0-9]", "", column.lower())
    try:
        return COLUMN_RENAMES[normalized]
    except KeyError as exc:
        raise ValueError(f"Unexpected column in UCI dataset: {column!r}") from exc


def _read_dataset_archive(archive_bytes: bytes) -> pd.DataFrame:
    with zipfile.ZipFile(BytesIO(archive_bytes)) as archive:
        csv_names = [name for name in archive.namelist() if name.lower().endswith(".csv")]
        if not csv_names:
            raise ValueError("The UCI download did not contain a CSV file")
        with archive.open(csv_names[0]) as csv_file:
            frame = pd.read_csv(csv_file)
    frame.columns = [_normalize_column_name(str(column)) for column in frame.columns]

    missing_columns = sorted(set(DATASET_COLUMNS) - set(frame.columns))
    if missing_columns:
        raise ValueError(
            "The UCI dataset is missing expected columns: " + ", ".join(missing_columns)
        )
    if frame.empty:
        raise ValueError("The UCI dataset contains no records")

    frame = frame.loc[:, DATASET_COLUMNS].copy()
    frame[TARGET_COLUMN] = (
        frame[TARGET_COLUMN]
        .astype(str)
        .str.strip()
        .str.lower()
        .map({"true": 1, "false": 0, "1": 1, "0": 0})
    )
    if frame[TARGET_COLUMN].isna().any():
        raise ValueError("The UCI purchase target contains values other than True or False")

    frame["weekend"] = (
        frame["weekend"].astype(str).str.strip().str.lower().map(
            {"true": "True", "false": "False", "1": "True", "0": "False"}
        )
    )
    if frame["weekend"].isna().any():
        raise ValueError("The UCI weekend column contains unexpected values")

    return frame


def ensure_dataset(
    path: str | Path = DATASET_PATH,
    *,
    source_url: str = DATASET_URL,
    overwrite: bool = False,
    timeout: int = 60,
) -> Path:
    """Ensure the official UCI CSV exists locally and return its path.

    Existing files are reused unless ``overwrite`` is true. Downloads are
    validated before the destination is replaced.
    """
    destination = Path(path)
    if destination.is_file() and not overwrite:
        LOGGER.info("Using existing dataset at %s", destination)
        return destination

    request = Request(source_url, headers={"User-Agent": "customer-purchase-prediction/1.0"})
    with urlopen(request, timeout=timeout) as response:
        frame = _read_dataset_archive(response.read())

    destination.parent.mkdir(parents=True, exist_ok=True)
    temporary_path = destination.with_suffix(destination.suffix + ".tmp")
    try:
        frame.to_csv(temporary_path, index=False)
        temporary_path.replace(destination)
    finally:
        if temporary_path.exists():
            temporary_path.unlink()
    LOGGER.info("Downloaded %s rows from %s to %s", len(frame), source_url, destination)
    return destination
```

src\data_preprocessing.py

```
"""Load, validate, split, and impute the UCI data with Spark."""

from __future__ import annotations

import logging
from typing import Any
from pyspark.sql import DataFrame
```

```
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType, StringType

from src.data_loader import (
    CATEGORICAL_COLUMNS,
    DATASET_COLUMNS,
    NUMERIC_COLUMNS,
    TARGET_COLUMN,
    ensure_dataset,
)

LOGGER = logging.getLogger(__name__)
ID_COLUMN = "session_id"


def load_dataset(spark: Any, path: str | None = None) -> DataFrame:
    """Load the project CSV as a Spark DataFrame."""
    dataset_path = ensure_dataset(path) if path is not None else ensure_dataset()
    frame = (
        spark.read.option("header", True)
        .option("inferSchema", True)
        .option("mode", "FAILFAST")
        .csv(str(dataset_path))
    )
    missing_columns = sorted(set(DATASET_COLUMNS) - set(frame.columns))
    if missing_columns:
        raise ValueError("Dataset is missing required columns: " + ", ".join(missing_columns))
    LOGGER.info("Loaded dataset from %s", dataset_path)
    return frame.select(*DATASET_COLUMNS)


def _clean_frame(frame: DataFrame) -> DataFrame:
    cleaned = frame.select(
        *[F.col(column).cast(DoubleType()).alias(column) for column in NUMERIC_COLUMNS],
        *[F.col(column).cast(StringType()).alias(column) for column in CATEGORICAL_COLUMNS],
        F.when(
            F.lower(F.col(TARGET_COLUMN).cast("string")).isin("true", "1"),
            F.lit(1.0),
        )
        .when(F.lower(F.col(TARGET_COLUMN).cast("string")).isin("false", "0"), F.lit(0.0))
        .otherwise(F.col(TARGET_COLUMN).cast(DoubleType()))
        .alias(TARGET_COLUMN),
    )
    cleaned = cleaned.dropDuplicates(DATASET_COLUMNS)

    nonnegative_columns = (
        "administrative",
        "administrative_duration",
        "informational",
        "informational_duration",
        "product_related",
        "product_related_duration",
        "page_values",
    )
    for column in nonnegative_columns:
        cleaned = cleaned.filter(F.col(column).isNull() | (F.col(column) >= 0))
    for column in ("bounce_rates", "exit_rates", "special_day"):
        cleaned = cleaned.filter(
            F.col(column).isNull() | F.col(column).between(0.0, 1.0)
        )
    cleaned = cleaned.filter(F.col(TARGET_COLUMN).isin(0.0, 1.0))
    return cleaned


def _impute_from_training(training: DataFrame, testing: DataFrame) -> tuple[DataFrame, DataFrame]:
    numeric_medians = training.agg(
        *[F.percentile_approx(F.col(column), 0.5).alias(column) for column in NUMERIC_COLUMNS]
    ).first()
    medians = {
        column: float(numeric_medians[column]) if numeric_medians[column] is not None else 0.0
        for column in NUMERIC_COLUMNS
    }

    categorical_modes: dict[str, str] = {}
    for column in CATEGORICAL_COLUMNS:
        mode = (
            training.filter(F.col(column).isNotNull() & (F.trim(F.col(column)) != ""))
            .groupBy(column)
            .count()
            .orderBy(F.desc("count"), F.asc(column))
            .first()
        )
        categorical_modes[column] = str(mode[column]) if mode is not None else "Unknown"

    def apply_imputation(data: DataFrame) -> DataFrame:
        result = data
        for column, median in medians.items():
            result = result.withColumn(
                column,
                F.when(
                    F.col(column).isNull() | F.isnan(F.col(column)),
                    F.lit(median),
                ).otherwise(F.col(column)),
            )
        for column, mode in categorical_modes.items():
```

```
            result = result.withColumn(
                column,
                F.when(
                    F.col(column).isNull() | (F.trim(F.col(column)) == ""),
                    F.lit(mode),
                ).otherwise(F.col(column)),
            )
        return result

    return apply_imputation(training), apply_imputation(testing)


def preprocess_data(
    frame: DataFrame,
    train_ratio: float = 0.8,
    random_seed: int = 42,
) -> tuple[DataFrame, DataFrame, dict[str, Any]]:
    """Clean and split the dataset; fit imputation values on training data only."""
    if not 0 < train_ratio < 1:
        raise ValueError("train_ratio must be strictly between 0 and 1")
    cleaned = _clean_frame(frame).cache()
    cleaned_count = cleaned.count()
    if cleaned_count < 2:
        raise ValueError("At least two valid dataset rows are required")

    # UCI provides one session per user but no record identifier; this is a generated row ID.
    ordered = cleaned.withColumn(ID_COLUMN, F.monotonically_increasing_id()).cache()
    ordered.count()
    train, test = ordered.randomSplit([train_ratio, 1 - train_ratio], seed=random_seed)
    train, test = train.cache(), test.cache()
    train_count, test_count = train.count(), test.count()
    if train_count == 0 or test_count == 0:
        raise ValueError("The train/test split produced an empty partition")
    train, test = _impute_from_training(train, test)
    train, test = train.cache(), test.cache()
    train_count, test_count = train.count(), test.count()

    null_counts = {
        column: frame.filter(F.col(column).isNull()).count()
        for column in frame.columns
    }
    class_distribution = {
        str(row[TARGET_COLUMN]): int(row["count"])
        for row in cleaned.groupBy(TARGET_COLUMN).count().collect()
    }
    summary = {
        "rows_before_cleaning": frame.count(),
        "rows_after_cleaning": cleaned_count,
        "columns": len(frame.columns),
        "column_names": list(frame.columns),
        "null_counts": null_counts,
        "class_distribution": class_distribution,
        "train_rows": train_count,
        "test_rows": test_count,
        "train_ratio": train_ratio,
        "random_seed": random_seed,
    }
    LOGGER.info(
        "Prepared %s rows (%s train, %s test)",
        cleaned_count,
        summary["train_rows"],
        summary["test_rows"],
    )
    return train, test, summary
```

src\feature_engineering.py

```
"""Shared Spark ML feature transformations used for training and prediction."""

from __future__ import annotations

from pyspark.ml import Pipeline
from pyspark.ml.base import Estimator, Transformer
from pyspark.ml.feature import OneHotEncoder, StandardScaler, StringIndexer, VectorAssembler

from src.data_loader import CATEGORICAL_COLUMNS, NUMERIC_COLUMNS, TARGET_COLUMN

LABEL_COLUMN = "label"
FEATURES_COLUMN = "features"
SCALED_FEATURES_COLUMN = "scaled_features"


def build_feature_stages() -> list[Estimator | Transformer]:
    """Build fit-on-training transformations; saved pipelines reuse them unchanged."""
    index_columns = [f"{column}_index" for column in CATEGORICAL_COLUMNS]
    encoded_columns = [f"{column}_encoded" for column in CATEGORICAL_COLUMNS]
    stages: list[Estimator | Transformer] = [
        StringIndexer(
            inputCol=column,
            outputCol=index_column,
            handleInvalid="keep",
        )
```

```
        for column, index_column in zip(CATEGORICAL_COLUMNS, index_columns)
    ]
    stages.extend(
        [
            OneHotEncoder(
                inputCols=index_columns,
                outputCols=encoded_columns,
                handleInvalid="keep",
                dropLast=False,
            ),
            VectorAssembler(
                inputCols=[*NUMERIC_COLUMNS, *encoded_columns],
                outputCol=FEATURES_COLUMN,
                handleInvalid="keep",
            ),
            StandardScaler(
                inputCol=FEATURES_COLUMN,
                outputCol=SCALED_FEATURES_COLUMN,
                withMean=True,
                withStd=True,
            ),
        ]
    )
    return stages


def prepare_model_frame(frame):
    """Alias the project target to Spark ML's conventional label column."""
    return frame.withColumnRenamed(TARGET_COLUMN, LABEL_COLUMN)


def build_pipeline(classifier: Estimator) -> Pipeline:
    """Create one complete preprocessing-and-classification pipeline."""
    return Pipeline(stages=[*build_feature_stages(), classifier])
```

src\model_evaluation.py

```
"""Evaluate saved Spark predictions and produce dashboard/report artifacts."""

from __future__ import annotations

import json
import logging
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from pyspark.ml.evaluation import BinaryClassificationEvaluator, MulticlassClassificationEvaluator

LOGGER = logging.getLogger(__name__)
METRIC_EXPLANATIONS = {
    "accuracy": "Share of all test sessions whose purchase status was classified correctly.",
    "precision": "Share of sessions predicted as purchases that were actual purchases.",
    "recall": "Share of actual purchase sessions that the model correctly identified.",
    "f1": "Harmonic mean of purchase precision and recall.",
    "roc_auc": "Ability to rank purchase sessions above non-purchase sessions across thresholds.",
}


def _calculate_metrics(predictions) -> dict[str, float]:
    multiclass = MulticlassClassificationEvaluator(
        labelCol="label",
        predictionCol="prediction",
    )
    metrics = {
        "accuracy": multiclass.setMetricName("accuracy").evaluate(predictions),
        "precision": multiclass.setMetricName("precisionByLabel").setMetricLabel(1.0).evaluate(
            predictions
        ),
        "recall": multiclass.setMetricName("recallByLabel").setMetricLabel(1.0).evaluate(
            predictions
        ),
        "f1": multiclass.setMetricName("fMeasureByLabel").setMetricLabel(1.0).evaluate(
            predictions
        ),
        "roc_auc": BinaryClassificationEvaluator(
            labelCol="label",
            rawPredictionCol="rawPrediction",
            metricName="areaUnderROC",
        ).evaluate(predictions),
    }
    return {name: float(value) for name, value in metrics.items()}


def _prediction_rows(predictions) -> pd.DataFrame:
    return predictions.select("label", "prediction", "probability").toPandas().assign(
        purchase_probability=lambda frame: frame["probability"].map(lambda vector: float(vector[1]))
```

```
    )


def _save_plots(
    results: dict[str, dict[str, Any]],
    metric_results: dict[str, dict[str, float]],
    best_model: str,
    output_directory: Path,
) -> None:
    best_rows = _prediction_rows(results[best_model]["predictions"])
    best_rows["label"] = best_rows["label"].astype(int)
    best_rows["prediction"] = best_rows["prediction"].astype(int)

    confusion = pd.crosstab(
        best_rows["label"],
        best_rows["prediction"],
        dropna=False,
    ).reindex(index=[0, 1], columns=[0, 1], fill_value=0)
    figure, axis = plt.subplots(figsize=(6, 5))
    sns.heatmap(
        confusion,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=["No purchase", "Purchase"],
        yticklabels=["No purchase", "Purchase"],
        ax=axis,
    )
    axis.set(title=f"Confusion Matrix - {best_model}", xlabel="Predicted", ylabel="Actual")
    figure.tight_layout()
    figure.savefig(output_directory / "confusion_matrix.png", dpi=160)
    plt.close(figure)

    from sklearn.metrics import roc_curve

    false_positive_rate, true_positive_rate, _ = roc_curve(
        best_rows["label"],
        best_rows["purchase_probability"],
    )
    figure, axis = plt.subplots(figsize=(7, 5))
    axis.plot(false_positive_rate, true_positive_rate, label=f"{best_model} (AUC
    {metric_results[best_model]['roc_auc']:.3f})")
    axis.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random classifier")
    axis.set(title="ROC Curve", xlabel="False positive rate", ylabel="True positive rate")
    axis.legend(loc="lower right")
    figure.tight_layout()
    figure.savefig(output_directory / "roc_curve.png", dpi=160)
    plt.close(figure)

    metric_frame = pd.DataFrame(metric_results).T
    axis = metric_frame.plot(kind="bar", figsize=(10, 6), ylim=(0, 1), rot=0)
    axis.set(title="Model Performance Comparison", ylabel="Score")
    axis.legend(loc="lower right")
    axis.figure.tight_layout()
    axis.figure.savefig(output_directory / "model_comparison.png", dpi=160)
    plt.close(axis.figure)

    actual_predicted = (
        best_rows.groupby(["label", "prediction"]).size().unstack(fill_value=0).reindex(
            index=[0, 1], columns=[0, 1], fill_value=0
        )
    )
    axis = actual_predicted.plot(kind="bar", figsize=(7, 5), rot=0)
    axis.set(
        title=f"Actual vs Predicted Sessions - {best_model}",
        xlabel="Actual outcome (0 = no purchase, 1 = purchase)",
        ylabel="Number of sessions",
    )
    axis.legend(["Predicted no purchase", "Predicted purchase"])
    axis.figure.tight_layout()
    axis.figure.savefig(output_directory / "actual_vs_predicted.png", dpi=160)
    plt.close(axis.figure)


def evaluate_models(
    results: dict[str, dict[str, Any]],
    output_directory: str | Path,
) -> dict[str, Any]:
    """Calculate all requested metrics and save JSON plus diagnostic plots."""
    if not results:
        raise ValueError("At least one trained model is required for evaluation")
    destination = Path(output_directory)
    destination.mkdir(parents=True, exist_ok=True)

    metrics = {
        model_name: _calculate_metrics(result["predictions"])
        for model_name, result in results.items()
    }
    best_model = max(
        metrics,
        key=lambda name: (metrics[name]["f1"], metrics[name]["roc_auc"], metrics[name]["accuracy"]),
    )
    _save_plots(results, metrics, best_model, destination)
    test_rows = results[best_model]["predictions"].count()
    report = {
```

```
        "dataset": "UCI Online Shoppers Purchasing Intention Dataset",
        "test_rows": test_rows,
        "positive_label": 1,
        "best_model": best_model,
        "selection_metric": "F1-score for the purchase class; ROC-AUC and accuracy break ties",
        "models": metrics,
        "metric_explanations": METRIC_EXPLANATIONS,
        "artifacts": {
            "confusion_matrix": "confusion_matrix.png",
            "roc_curve": "roc_curve.png",
            "model_comparison": "model_comparison.png",
            "actual_vs_predicted": "actual_vs_predicted.png",
        },
    }
    metrics_path = destination / "model_metrics.json"
    metrics_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    LOGGER.info("Best model: %s; metrics saved to %s", best_model, metrics_path)
    return report
```

src\model_training.py

```
"""Fit and persist the required Spark binary classifiers."""

from __future__ import annotations

import logging
from pathlib import Path
from typing import Any

from pyspark.ml.classification import GBTClassifier, LogisticRegression
from pyspark.ml.pipeline import PipelineModel

from src.feature_engineering import SCALED_FEATURES_COLUMN, build_pipeline, prepare_model_frame

LOGGER = logging.getLogger(__name__)
MODEL_DIRECTORY = Path(__file__).resolve().parent.parent / "models"


def train_models(
    training_data,
    test_data,
    models_directory: str | Path = MODEL_DIRECTORY,
    random_seed: int = 42,
) -> dict[str, dict[str, Any]]:
    """Train Logistic Regression and GBT pipelines using training data only."""
    model_directory = Path(models_directory)
    model_directory.mkdir(parents=True, exist_ok=True)
    train_frame = prepare_model_frame(training_data)
    test_frame = prepare_model_frame(test_data)

    classifiers = {
        "Logistic Regression": LogisticRegression(
            labelCol="label",
            featuresCol=SCALED_FEATURES_COLUMN,
            maxIter=100,
            regParam=0.01,
            elasticNetParam=0.0,
        ),
        "Gradient-Boosted Trees": GBTClassifier(
            labelCol="label",
            featuresCol=SCALED_FEATURES_COLUMN,
            maxIter=30,
            maxDepth=5,
            seed=random_seed,
        ),
    }

    results: dict[str, dict[str, Any]] = {}
    for model_name, classifier in classifiers.items():
        LOGGER.info("Training %s", model_name)
        fitted_pipeline = build_pipeline(classifier).fit(train_frame)
        predictions = fitted_pipeline.transform(test_frame).cache()
        predictions.count()
        model_path = model_directory / model_name.lower().replace(" ", "_").replace("-", "")
        fitted_pipeline.write().overwrite().save(str(model_path))
        results[model_name] = {
            "model": fitted_pipeline,
            "predictions": predictions,
            "model_path": str(model_path),
        }
        LOGGER.info("Saved %s pipeline to %s", model_name, model_path)
    return results


def load_saved_model(path: str | Path) -> PipelineModel:
    """Load a persisted feature-and-classification pipeline."""
    return PipelineModel.load(str(path))
```

src\prediction.py

```
"""Export holdout predictions and support single-session inference."""

from __future__ import annotations

import logging
from pathlib import Path
from typing import Any

from src.data_loader import CATEGORICAL_COLUMNS, NUMERIC_COLUMNS
from src.data_preprocessing import ID_COLUMN
from src.feature_engineering import prepare_model_frame
from src.model_training import load_saved_model

LOGGER = logging.getLogger(__name__)
OUTPUT_PATH = Path(__file__).resolve().parent.parent / "outputs" / "purchase_predictions.csv"


def export_predictions(
    model_path: str | Path,
    test_data,
    output_path: str | Path = OUTPUT_PATH,
) -> Path:
    """Transform holdout sessions with the persisted pipeline and save predictions."""
    model = load_saved_model(model_path)
    transformed = model.transform(prepare_model_frame(test_data))
    selected_columns = [
        ID_COLUMN,
        *NUMERIC_COLUMNS,
        *CATEGORICAL_COLUMNS,
        "label",
        "prediction",
        "probability",
    ]
    records = transformed.select(*selected_columns).toPandas()
    records["purchase_probability"] = records["probability"].map(
        lambda value: float(value[1])
    )
    records["customer_id"] = records[ID_COLUMN].map(
        lambda value: f"session-{int(value):06d}"
    )
    records["actual_purchase"] = records["label"].astype(int)
    records["predicted_purchase"] = records["prediction"].astype(int)
    records["correct_prediction"] = (
        records["actual_purchase"] == records["predicted_purchase"]
    )
    records = records[
        [
            "customer_id",
            *NUMERIC_COLUMNS,
            *CATEGORICAL_COLUMNS,
            "actual_purchase",
            "predicted_purchase",
            "purchase_probability",
            "correct_prediction",
        ]
    ]
    destination = Path(output_path)
    destination.parent.mkdir(parents=True, exist_ok=True)
    records.to_csv(destination, index=False)
    LOGGER.info("Saved %s test predictions to %s", len(records), destination)
    return destination


def predict_session(model, spark, features: dict[str, Any]) -> dict[str, Any]:
    """Predict a single session using the same persisted feature pipeline."""
    expected = {*NUMERIC_COLUMNS, *CATEGORICAL_COLUMNS}
    missing = expected - features.keys()
    extra = features.keys() - expected
    if missing or extra:
        raise ValueError(
            f"Feature fields do not match model schema; missing={sorted(missing)}, "
            f"unexpected={sorted(extra)}"
        )
    row = {
        column: (
            str(features[column])
            if column in CATEGORICAL_COLUMNS
            else float(features[column])
        )
        for column in [*NUMERIC_COLUMNS, *CATEGORICAL_COLUMNS]
    }
    prediction = model.transform(spark.createDataFrame([row])).first()
    return {
        "predicted_purchase": int(prediction["prediction"]),
        "purchase_probability": float(prediction["probability"][1]),
    }
```

src\spark_session.py

```
"""Shared configuration for the project's local PySpark session."""

from __future__ import annotations
```

```
import logging
import os
import shutil
import sys
from pathlib import Path

from pyspark.sql import SparkSession

LOGGER = logging.getLogger(__name__)


def _configure_windows_hadoop() -> None:
    hadoop_home = os.environ.get("HADOOP_HOME")
    if hadoop_home and (Path(hadoop_home) / "bin" / "winutils.exe").is_file():
        return

    winutils = shutil.which("winutils.exe")
    if winutils:
        hadoop_home = str(Path(winutils).resolve().parent.parent)
        os.environ["HADOOP_HOME"] = hadoop_home
        os.environ["hadoop.home.dir"] = hadoop_home
        LOGGER.info("Using winutils from %s", winutils)
        return

    raise RuntimeError(
        "Windows Spark model persistence requires winutils.exe. Install Hadoop "
        "winutils matching the Hadoop version bundled with PySpark, set HADOOP_HOME "
        "to its parent directory, and add HADOOP_HOME\\bin to PATH."
    )


def create_spark_session(app_name: str = "CustomerPurchasePrediction") -> SparkSession:
    """Create or reuse a Windows-compatible local Spark session."""
    if os.name == "nt":
        _configure_windows_hadoop()
    os.environ["PYSPARK_PYTHON"] = sys.executable
    os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
    builder = (
        SparkSession.builder.appName(app_name)
        .master("local[*]")
        .config("spark.driver.memory", "4g")
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.sql.session.timeZone", "UTC")
    )
    spark = builder.getOrCreate()
    spark.sparkContext.setLogLevel("WARN")
    LOGGER.info("Started Spark %s in %s mode", spark.version, spark.sparkContext.master)
    return spark
```

dashboard\app.py

```
"""Streamlit analytics dashboard for the saved UCI model artifacts."""

from __future__ import annotations

import json
import sys
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st
from pyspark.errors import PySparkException

PROJECT_ROOT = Path(__file__).resolve().parent.parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from src.data_loader import (  # noqa: E402
    CATEGORICAL_COLUMNS,
    DATASET_PATH,
    NUMERIC_COLUMNS,
    UCI_DATASET_PAGE,
)
from src.model_training import load_saved_model  # noqa: E402
from src.prediction import predict_session  # noqa: E402
from src.spark_session import create_spark_session  # noqa: E402

OUTPUT_DIR = PROJECT_ROOT / "outputs"
METRICS_PATH = OUTPUT_DIR / "model_metrics.json"
PREDICTIONS_PATH = OUTPUT_DIR / "purchase_predictions.csv"
MODELS_DIR = PROJECT_ROOT / "models"

st.set_page_config(
    page_title="Customer Purchase Behaviour Analytics",
    layout="wide",
)


@st.cache_data
def load_artifacts() -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    if not DATASET_PATH.is_file():
```

```
        raise FileNotFoundError(f"Dataset not found: {DATASET_PATH}")
    if not METRICS_PATH.is_file() or not PREDICTIONS_PATH.is_file():
        raise FileNotFoundError("Model outputs are missing. Run `python main.py` first.")
    dataset = pd.read_csv(DATASET_PATH)
    predictions = pd.read_csv(PREDICTIONS_PATH)
    metrics = json.loads(METRICS_PATH.read_text(encoding="utf-8"))
    return dataset, predictions, metrics


@st.cache_resource
def load_model_resources(model_directory: str):
    spark = create_spark_session("CustomerPurchaseDashboard")
    model = load_saved_model(model_directory)
    return spark, model


def _session_duration(frame: pd.DataFrame) -> pd.Series:
    return (
        frame["administrative_duration"]
        + frame["informational_duration"]
        + frame["product_related_duration"]
    )


def _add_behavior_groups(frame: pd.DataFrame) -> pd.DataFrame:
    result = frame.copy()
    result["session_duration"] = _session_duration(result)
    result["browsing_activity"] = pd.cut(
        result["product_related"],
        bins=[-float("inf"), 5, 20, float("inf")],
        labels=["Low (0-5)", "Medium (6-20)", "High (21+)"],
    ).astype(str)
    result["session_duration_category"] = pd.cut(
        result["session_duration"],
        bins=[-float("inf"), 300, 1800, float("inf")],
        labels=["Short (0-5 min)", "Medium (5-30 min)", "Long (>30 min)"],
    ).astype(str)
    result["purchase_status"] = result["purchase"].map(
        {0: "No purchase", 1: "Purchase", False: "No purchase", True: "Purchase"}
    )
    return result


try:
    customers, prediction_results, model_metrics = load_artifacts()
except (FileNotFoundError, pd.errors.ParserError, json.JSONDecodeError) as error:
    st.error(str(error))
    st.info("Generate the dataset, trained models, and outputs with `python main.py`.")
    st.stop()

customers = _add_behavior_groups(customers)
predictions = prediction_results.copy()
best_model_name = model_metrics["best_model"]
best_metrics = model_metrics["models"][best_model_name]

st.title("Customer Purchase Behaviour Analytics Dashboard")
st.caption(
    "Apache Spark MLlib classification using the UCI Online Shoppers Purchasing "
    "Intention Dataset. The source contains sessions, not customer age or prior "
    "purchase counts."
)
st.markdown(f"Dataset source: [{UCI_DATASET_PAGE}]({UCI_DATASET_PAGE})")

total_sessions = len(customers)
total_purchases = int((customers["purchase"] == 1).sum())
total_non_purchases = total_sessions - total_purchases
holdout_predicted_purchases = int((predictions["predicted_purchase"] == 1).sum())
conversion_rate = total_purchases / total_sessions if total_sessions else 0

kpi_columns = st.columns(6)
kpis = [
    ("Total sessions", f"{total_sessions:,}"),
    ("Actual purchases", f"{total_purchases:,}"),
    ("No purchases", f"{total_non_purchases:,}"),
    ("Actual conversion", f"{conversion_rate:.1%}"),
    ("Holdout predicted purchases", f"{holdout_predicted_purchases:,}"),
    ("Holdout model accuracy", f"{best_metrics['accuracy']:.1%}"),
]
for column, (label, value) in zip(kpi_columns, kpis):
    column.metric(label, value)

overview, segments, performance, predictions_tab, inference = st.tabs(
    ["Purchase analysis", "Customer segmentation", "Model performance", "Predictions", "Try a prediction"]
)

with overview:
    st.subheader("Observed purchase outcomes")
    left, right = st.columns(2)
    outcome_counts =
    customers["purchase_status"].value_counts().rename_axis("Outcome").reset_index(name="Sessions")
    left.plotly_chart(
        px.pie(outcome_counts, names="Outcome", values="Sessions", hole=0.45, title="Purchase vs no purchase"),
        use_container_width=True,
    )
    right.plotly_chart(
```

```
        px.bar(outcome_counts, x="Outcome", y="Sessions", color="Outcome", title="Session outcome counts"),
        use_container_width=True,
    )
    st.info(
        "Age-group and past-purchase charts are not available: those fields are not in "
        "the source. `VisitorType` is shown as a new/returning-visitor descriptor, not "
        "as a count of previous purchases."
    )

with segments:
    st.subheader("Conversion by observed session behaviour")
    selected_group = st.selectbox(
        "Segment sessions by",
        ["Browsing activity", "Session duration", "Visitor type (proxy only)", "Month"],
    )
    group_column = {
        "Browsing activity": "browsing_activity",
        "Session duration": "session_duration_category",
        "Visitor type (proxy only)": "visitor_type",
        "Month": "month",
    }[selected_group]
    grouped = (
        customers.groupby(group_column, observed=False)["purchase"]
        .agg(sessions="size", purchases="sum", conversion_rate="mean")
        .reset_index()
        .rename(columns={group_column: "Segment"})
    )
    left, right = st.columns(2)
    left.plotly_chart(
        px.bar(grouped, x="Segment", y="sessions", color="Segment", title="Sessions per segment"),
        use_container_width=True,
    )
    right.plotly_chart(
        px.bar(
            grouped,
            x="Segment",
            y="conversion_rate",
            color="Segment",
            title="Observed purchase conversion rate",
            labels={"conversion_rate": "Conversion rate"},
        ),
        use_container_width=True,
    )
    st.caption(
        "Session duration is derived as the sum of the three page-category duration fields. "
        "Browsing activity uses ProductRelated page views. Visitor type is not purchase history."
    )

with performance:
    st.subheader(f"Holdout evaluation - {best_model_name}")
    metric_names = ["accuracy", "precision", "recall", "f1", "roc_auc"]
    metric_columns = st.columns(len(metric_names))
    for column, metric_name in zip(metric_columns, metric_names):
        column.metric(metric_name.replace("_", " ").title(), f"{best_metrics[metric_name]:.3f}")
    st.caption(model_metrics["selection_metric"])
    comparison = pd.DataFrame(model_metrics["models"]).T.reset_index(names="Model")
    st.plotly_chart(
        px.bar(
            comparison.melt(id_vars="Model", var_name="Metric", value_name="Score"),
            x="Metric",
            y="Score",
            color="Model",
            barmode="group",
            title="Model comparison on the same holdout set",
            range_y=[0, 1],
        ),
        use_container_width=True,
    )
    figure_columns = st.columns(2)
    for column, filename, title in zip(
        figure_columns,
        ["confusion_matrix.png", "roc_curve.png"],
        ["Confusion matrix", "ROC curve"],
    ):
        image_path = OUTPUT_DIR / filename
        if image_path.is_file():
            column.image(str(image_path), caption=title, use_container_width=True)
    st.image(
        str(OUTPUT_DIR / "actual_vs_predicted.png"),
        caption="Actual versus predicted test-session counts",
        use_container_width=True,
    )

with predictions_tab:
    st.subheader("Holdout prediction results")
    correct_count = int(predictions["correct_prediction"].astype(bool).sum())
    incorrect_count = len(predictions) - correct_count
    st.write(f"Correct: **{correct_count:,}** | Incorrect: **{incorrect_count:,}**")
    st.plotly_chart(
        px.histogram(
            predictions,
            x="actual_purchase",
            color="predicted_purchase",
            barmode="group",
            title="Actual versus predicted outcomes",
```

```
            labels={"actual_purchase": "Actual purchase", "predicted_purchase": "Predicted purchase"},
        ),
        use_container_width=True,
    )
    st.dataframe(predictions, use_container_width=True, hide_index=True)
    st.download_button(
        "Download holdout predictions",
        data=predictions.to_csv(index=False).encode("utf-8"),
        file_name="purchase_predictions.csv",
        mime="text/csv",
    )

with inference:
    st.subheader("Estimate purchase probability for one session")
    st.warning(
        "Enter fields that exist in the source dataset. Age and prior purchase count are "
        "not available and are intentionally not requested."
    )
    with st.form("session_prediction_form"):
        numeric_values = {}
        columns = st.columns(2)
        integer_columns = {
            "administrative",
            "informational",
            "product_related",
            "operating_systems",
            "browser",
            "region",
            "traffic_type",
        }
        fractional_columns = {"bounce_rates", "exit_rates", "special_day"}
        for index, feature in enumerate(NUMERIC_COLUMNS):
            minimum = float(customers[feature].min())
            maximum = float(customers[feature].max())
            default = float(customers[feature].median())
            step = 0.01 if feature in fractional_columns else (
                1.0 if feature in integer_columns else 10.0
            )
            numeric_values[feature] = columns[index % 2].number_input(
                feature.replace("_", " ").title(),
                min_value=minimum,
                max_value=maximum,
                value=min(max(default, minimum), maximum),
                step=step,
                key=f"input_{feature}",
            )
        categorical_values = {}
        for index, feature in enumerate(CATEGORICAL_COLUMNS):
            choices = sorted(customers[feature].dropna().astype(str).unique().tolist())
            if "Unknown" not in choices:
                choices.append("Unknown")
            categorical_values[feature] = columns[index % 2].selectbox(
                feature.replace("_", " ").title(),
                choices,
                key=f"input_{feature}",
            )
        submitted = st.form_submit_button("Predict purchase likelihood")

    if submitted:
        model_directory = MODELS_DIR / best_model_name.lower().replace(" ", "_").replace("-", "")
        try:
            spark, model = load_model_resources(str(model_directory))
            prediction = predict_session(
                model,
                spark,
                {**numeric_values, **categorical_values},
            )
        except (ValueError, OSError, PySparkException) as error:
            st.error(f"Prediction failed: {error}")
        else:
            outcome = "Likely purchase" if prediction["predicted_purchase"] else "Likely no purchase"
            st.success(f"{outcome} - purchase probability {prediction['purchase_probability']:.1%}")
```

report\generate_report.py

```
"""Generate the academic project PDF from real run artifacts."""

from __future__ import annotations

import json
import textwrap
from pathlib import Path
from typing import Any

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.platypus import (
    BaseDocTemplate,
    Image,
```

```
    PageTemplate,
    Paragraph,
    Preformatted,
    Spacer,
    Table,
    Frame,
    PageBreak,
)
from reportlab.platypus.tableofcontents import TableOfContents

PROJECT_ROOT = Path(__file__).resolve().parent.parent
OUTPUTS = PROJECT_ROOT / "outputs"
SCREENSHOT_CHECKLIST = PROJECT_ROOT / "screenshots" / "CHECKLIST.md"


class ProjectReportTemplate(BaseDocTemplate):
    def __init__(self, filename: str | Path, styles: dict[str, ParagraphStyle]) -> None:
        super().__init__(
            str(filename),
            pagesize=letter,
            rightMargin=0.72 * inch,
            leftMargin=0.72 * inch,
            topMargin=0.78 * inch,
            bottomMargin=0.68 * inch,
            title="Predicting Customer Purchase Behaviour Using Apache Spark and Machine Learning",
            author="INT315 - Cluster Computing (CA2)",
        )
        self._styles = styles
        frame = Frame(
            self.leftMargin,
            self.bottomMargin,
            self.width,
            self.height,
            id="normal",
        )
        self.addPageTemplates(
            [PageTemplate(id="main", frames=frame, onPage=self._draw_page_number)]
        )
        self._heading_count = 0

    def beforeDocument(self) -> None:
        self._heading_count = 0

    @staticmethod
    def _draw_page_number(canvas, document) -> None:
        canvas.saveState()
        canvas.setFont("Helvetica", 8)
        canvas.setFillColor(colors.HexColor("#5B6573"))
        canvas.drawCentredString(letter[0] / 2, 0.38 * inch, str(document.page))
        canvas.restoreState()

    def afterFlowable(self, flowable) -> None:
        if isinstance(flowable, Paragraph) and flowable.style.name == "Heading1":
            self._heading_count += 1
            title = flowable.getPlainText()
            key = f"section-{self._heading_count}"
            self.canv.bookmarkPage(key)
            self.notify("TOCEntry", (0, title, self.page, key))


def _styles() -> dict[str, ParagraphStyle]:
    styles = getSampleStyleSheet()
    styles.add(
        ParagraphStyle(
            name="TitlePage",
            parent=styles["Title"],
            fontName="Helvetica-Bold",
            fontSize=22,
            leading=29,
            alignment=TA_CENTER,
            textColor=colors.HexColor("#17365D"),
            spaceAfter=18,
        )
    )
    styles["Heading1"].fontSize = 15
    styles["Heading1"].leading = 19
    styles["Heading1"].textColor = colors.HexColor("#17365D")
    styles["Heading1"].spaceBefore = 13
    styles["Heading1"].spaceAfter = 8
    styles["Heading2"].fontSize = 11
    styles["Heading2"].leading = 14
    styles["Heading2"].textColor = colors.HexColor("#2F5597")
    styles["Heading2"].spaceBefore = 9
    styles["Heading2"].spaceAfter = 5
    styles.add(
        ParagraphStyle(
            name="BodyAcademic",
            parent=styles["BodyText"],
            fontSize=9.5,
            leading=14,
            spaceAfter=7,
        )
    )
    styles.add(
        ParagraphStyle(
```

```
            name="CodeSmall",
            fontName="Courier",
            fontSize=5.8,
            leading=7.1,
            leftIndent=5,
            rightIndent=5,
            backColor=colors.HexColor("#F3F5F7"),
        )
    )
    styles.add(ParagraphStyle(name="TOCHeading", parent=styles["Heading1"]))
    styles.add(
        ParagraphStyle(
            name="TOCEntry0",
            fontName="Helvetica",
            fontSize=10,
            leading=15,
            leftIndent=8,
            firstLineIndent=-8,
            spaceBefore=3,
        )
    )
    return styles


def _paragraph(text: str, styles: dict[str, ParagraphStyle]) -> Paragraph:
    return Paragraph(text, styles["BodyAcademic"])


def _section(title: str, styles: dict[str, ParagraphStyle], story: list) -> None:
    story.append(Paragraph(title, styles["Heading1"]))


def _screenshot_placeholder(label: str) -> Table:
    table = Table(
        [[Paragraph(f"<b>Screenshot placeholder:</b> {label}<br/>Insert an authentic project screenshot here.",
    getSampleStyleSheet()["BodyText"])]],
        colWidths=[6.7 * inch],
        rowHeights=[0.72 * inch],
    )
    table.setStyle(
        [
            ("BOX", (0, 0), (-1, -1), 0.8, colors.HexColor("#7F8C9A")),
            ("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#F4F6F8")),
            ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
            ("LEFTPADDING", (0, 0), (-1, -1), 10),
        ]
    )
    return table


def _metric_table(evaluation: dict[str, Any]) -> Table:
    rows = [["Model", "Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"]]
    for model_name, metrics in evaluation["models"].items():
        rows.append(
            [
                model_name,
                *[f"{metrics[name]:.4f}" for name in ("accuracy", "precision", "recall", "f1", "roc_auc")],
            ]
        )
    table = Table(rows, repeatRows=1, colWidths=[1.55 * inch, 1.0 * inch, 0.9 * inch, 0.8 * inch, 0.8 * inch,
    0.9 * inch])
    table.setStyle(
        [
            ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#17365D")),
            ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
            ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
            ("GRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#ABB7C4")),
            ("ALIGN", (1, 1), (-1, -1), "CENTER"),
            ("FONTSIZE", (0, 0), (-1, -1), 8),
            ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#F2F5F8")]),
            ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ]
    )
    return table


def _source_files() -> list[Path]:
    paths = [PROJECT_ROOT / "main.py", *sorted((PROJECT_ROOT / "src").glob("*.py"))]
    paths.extend([PROJECT_ROOT / "dashboard" / "app.py", Path(__file__).resolve()])
    return [path for path in paths if path.is_file()]


def _appendix_code(path: Path, styles: dict[str, ParagraphStyle], story: list) -> None:
    relative_path = path.relative_to(PROJECT_ROOT)
    story.append(Paragraph(str(relative_path), styles["Heading2"]))
    wrapped_lines = []
    for line in path.read_text(encoding="utf-8").splitlines():
        wrapped_lines.extend(
            textwrap.wrap(
                line,
                width=112,
                subsequent_indent="    ",
                replace_whitespace=False,
                drop_whitespace=False,
            )
```

```
            or [""]
        )
    story.append(Preformatted("\n".join(wrapped_lines), styles["CodeSmall"]))
    story.append(Spacer(1, 8))


def generate_report(
    preprocessing_summary: dict[str, Any] | None = None,
    evaluation: dict[str, Any] | None = None,
    dataset_path: str | Path | None = None,
    output_path: str | Path | None = None,
) -> Path:
    """Create a report using actual run metrics; refuse missing execution results."""
    summary_path = OUTPUTS / "preprocessing_summary.json"
    metrics_path = OUTPUTS / "model_metrics.json"
    if preprocessing_summary is None:
        if not summary_path.is_file():
            raise FileNotFoundError("Run main.py before generating the report.")
        preprocessing_summary = json.loads(summary_path.read_text(encoding="utf-8"))
    if evaluation is None:
        if not metrics_path.is_file():
            raise FileNotFoundError("Model metrics are missing; run main.py first.")
        evaluation = json.loads(metrics_path.read_text(encoding="utf-8"))
    if not evaluation.get("models"):
        raise ValueError("The report requires actual metrics for at least one trained model")

    destination = Path(output_path) if output_path else PROJECT_ROOT / "report" / "project_report.pdf"
    destination.parent.mkdir(parents=True, exist_ok=True)
    styles = _styles()
    story: list[Any] = [
        Spacer(1, 1.2 * inch),
        Paragraph(
            "Predicting Customer Purchase Behaviour<br/>Using Apache Spark and Machine Learning",
            styles["TitlePage"],
        ),
        Spacer(1, 0.35 * inch),
        _paragraph("INT315 - Cluster Computing (CA2)", styles),
        Spacer(1, 0.2 * inch),
        _paragraph("Academic project report", styles),
        Spacer(1, 2.4 * inch),
        _paragraph(
            "Prepared with measured outputs from the local Spark execution. "
            "No unrun results or screenshots are represented as evidence.",
            styles,
        ),
        PageBreak(),
        Paragraph("Table of Contents", styles["TOCHeading"]),
    ]
    toc = TableOfContents()
    toc.levelStyles = [styles["TOCEntry0"]]
    story.extend([toc, PageBreak()])

    _section("1. Project Title", styles, story)
    story.append(_paragraph("Predicting Customer Purchase Behaviour Using Apache Spark and Machine Learning.",
    styles))

    _section("2. Problem Statement", styles, story)
    story.append(
        _paragraph(
            "E-commerce teams need to understand which browsing sessions are associated with a purchase. "
            "This project uses session-level website interaction data to classify whether a session ends "
            "with a transaction. The target is binary: 1 indicates that a purchase was recorded and 0 "
            "indicates no purchase.",
            styles,
        )
    )

    _section("3. Objective of the Project", styles, story)
    story.append(
        _paragraph(
            "The objective is to build a reproducible Spark workflow that loads and validates the public "
            "dataset, performs leakage-aware preprocessing, trains Logistic Regression and Gradient-Boosted "
            "Trees, evaluates the models on a held-out test set, exports session predictions, and presents "
            "the measured outputs in a dashboard.",
            styles,
        )
    )

    _section("4. Dataset Description", styles, story)
    story.append(
        _paragraph(
            "The dataset contains online-shopping sessions rather than demographic customer profiles. "
            "The public UCI record describes 12,330 sessions, with one session per user in the sampled "
            "period. The project keeps the source features and uses a generated session identifier only "
            "for output traceability; that identifier is not a source customer ID and is excluded from training.
     "
            f"The local project copy used for this run is {Path(dataset_path).name if dataset_path else
    'customer_behaviour.csv'}.",
            styles,
        )
    )
    column_rows = [
        ["Field(s)", "Meaning"],
        ["Administrative, Informational, ProductRelated", "Counts of visited pages in each page category."],
        ["*_Duration", "Time spent in each corresponding page category during the session."],
```

```
    ["BounceRates, ExitRates, PageValues", "Web analytics measures associated with the session."],
    ["SpecialDay", "Proximity measure for a special shopping day."],
    ["Month, OperatingSystems, Browser, Region, TrafficType", "Session time and technical/traffic
categories."],
    ["VisitorType, Weekend", "New/returning visitor category and weekend indicator."],
    [        "Revenue -> purchase", "Binary outcome; True maps to 1 (purchase), False to 0."],
]
schema_table = Table(column_rows, repeatRows=1, colWidths=[2.5 * inch, 4.2 * inch])
schema_table.setStyle(
    [
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#17365D")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("GRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#ABB7C4")),
        ("FONTSIZE", (0, 0), (-1, -1), 8),
        ("VALIGN", (0, 0), (-1, -1), "TOP"),
    ]
)
story.append(schema_table)
story.append(
    _paragraph(
        "Limitations: the source has no age/age-group, explicit session-duration field, or previous "
        "purchase-count field. Session duration in this project is derived from the sum of the three "
        "page-category duration features. VisitorType distinguishes new from returning visitors but "
        "is not a count of past purchases. The project does not fabricate the missing attributes.",
        styles,
    )
)

_section("5. Dataset Source", styles, story)
story.append(
    _paragraph(
        "Source: Sakar, C. and Kastro, Y. (2018), Online Shoppers Purchasing Intention Dataset, "
        "UCI Machine Learning Repository. DOI: 10.24432/C5F88Q. "
        "Dataset page: https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset.
 "
        "Direct archive:
https://archive.ics.uci.edu/static/public/468/online+shoppers+purchasing+intention+dataset.zip. "
        "The dataset is distributed under CC BY 4.0; attribution is required.",
        styles,
    )
)

_section("6. Tools and Technologies Used", styles, story)
story.append(
    _paragraph(
        "Python 3, Apache Spark in local[*] mode, PySpark SQL and MLlib, Pandas, NumPy, "
        "Matplotlib, Seaborn, scikit-learn for ROC curve coordinates, Streamlit, Plotly, "
        "and ReportLab. Local mode uses one machine and is not represented as a multi-node cluster run.",
        styles,
    )
)

_section("7. Implementation and Methodology", styles, story)
story.append(
    _paragraph(
        f"On this execution, {preprocessing_summary['rows_before_cleaning']:,} source rows were loaded "
        f"and {preprocessing_summary['rows_after_cleaning']:,} remained after validation and duplicate "
        "removal. The split used "
        f"{preprocessing_summary['train_ratio']:.0%} training data and a fixed seed "
        f"of {preprocessing_summary['random_seed']}. Numerical medians and categorical modes are "
        "estimated from training rows only and applied to both partitions. StringIndexers, OneHotEncoder, "
        "VectorAssembler, StandardScaler, and the classifier are fitted together in each Spark ML
Pipeline.",
        styles,
    )
)
story.append(
    _paragraph(
        "Logistic Regression estimates a linear decision boundary for the binary target. "
        "Gradient-Boosted Trees combine shallow decision trees sequentially to model nonlinear "
        "feature interactions. Both pipelines are fit only on training data and produce predictions "
        "for the same held-out test partition.",
        styles,
    )
)
story.append(
    _paragraph(
        f"Observed source target distribution: {preprocessing_summary['class_distribution']}. "
        "The identifiers used in exported predictions are generated session row IDs, not customer "
        "identities from UCI.",
        styles,
    )
)

_section("8. Complete Source Code", styles, story)
story.append(
    _paragraph(
        "The executable Python sources used for this project are included in the appendices of this PDF. "
        "The workspace files remain the authoritative editable source.",
        styles,
    )
)

_section("9. Execution and Output Screenshots", styles, story)
story.append(
```

```
    _paragraph(
        "Authentic screenshots are not generated by the report process. Insert screenshots captured "
        "from your own execution in the marked locations below and save the originals under screenshots/.",
        styles,
    )
)
for label in (
    "Project folder structure",
    "Dataset loaded into Spark and schema",
    "Missing-value analysis and preprocessing output",
    "Feature encoding and VectorAssembler",
    "Logistic Regression training",
    "Gradient-Boosted Trees training",
    "Model evaluation metrics",
    "Confusion matrix and ROC curve",
    "Actual-versus-predicted purchases",
    "Dashboard overview and customer segmentation",
    "Interactive session prediction",
    "Successful end-to-end execution",
):
    story.extend([_screenshot_placeholder(label), Spacer(1, 5)])

_section("10. Results and Analysis", styles, story)
story.append(_metric_table(evaluation))
story.append(Spacer(1, 8))
story.append(
    _paragraph(
        f"Selected model: {evaluation['best_model']}. Selection prioritizes purchase-class F1-score, "
        "with ROC-AUC and accuracy as tie-breakers. Accuracy measures the fraction of all correct "
        "classifications; precision measures how many predicted purchases were actual purchases; "
        "recall measures how many actual purchases were detected; F1 balances precision and recall; "
        "ROC-AUC measures ranking quality over decision thresholds. The following values are read "
        "from Spark evaluation on this run's holdout predictions and must not be interpreted as "
        "guarantees of future performance.",
        styles,
    )
)
for filename, caption in (
    ("model_comparison.png", "Performance comparison of the two Spark models."),
    ("confusion_matrix.png", "Best-model confusion matrix on the held-out test sessions."),
    ("roc_curve.png", "Best-model ROC curve on the held-out test sessions."),
    ("actual_vs_predicted.png", "Actual and predicted outcome counts for the holdout set."),
):
    figure_path = OUTPUTS / filename
    if figure_path.is_file():
        story.extend(
            [
                Spacer(1, 8),
                Paragraph(caption, styles["BodyAcademic"]),
                Image(str(figure_path), width=5.6 * inch, height=3.1 * inch),
            ]
        )
    else:
        story.append(_screenshot_placeholder(caption))

_section("11. Conclusion", styles, story)
story.append(
    _paragraph(
        "This project demonstrates a complete local Spark ML workflow using a public, documented "
        "e-commerce session dataset. The trained models and test metrics are generated by execution, "
        "and the Streamlit dashboard reads persisted artifacts rather than retraining on page refresh. "
        "Future work may validate generalization on newer sessions, tune thresholds for business costs, "
        "and use a consented dataset that actually includes demographic and prior-purchase attributes.",
        styles,
    )
)

_section("12. Plagiarism Report", styles, story)
story.append(
    _paragraph(
        "A plagiarism/similarity report must be generated separately using a university-approved "
        "plagiarism-checking tool. No similarity percentage is asserted in this document. Attach the "
        "institutionally generated report here if required.",
        styles,
    )
)
story.append(_screenshot_placeholder("University plagiarism-checking report (attach separately)"))

_section("13. References", styles, story)
story.append(
    _paragraph(
        "Sakar, C. and Kastro, Y. (2018). Online Shoppers Purchasing Intention Dataset. "
        "UCI Machine Learning Repository. https://doi.org/10.24432/C5F88Q.<br/>"
        "Apache Spark documentation: https://spark.apache.org/docs/latest/<br/>"
        "UCI dataset page:
https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset",
        styles,
    )
)

story.append(PageBreak())
story.append(Paragraph("Appendix A - Complete Python Source Code", styles["Heading1"]))
for source_file in _source_files():
    _appendix_code(source_file, styles, story)
report = ProjectReportTemplate(destination, styles)
```

```
    report.multiBuild(story)
    return destination


if __name__ == "__main__":
    print(generate_report())
```